Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 5

Threads, Workers and Signals

Lab 5 guide: start workers, wait for each one to finish, and tell completion apart from protecting shared data.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create POSIX threads, and check the return codes of the API.
2. Join each worker before using its result, and explain shared versus separate address spaces.
3. Use a bounded run to find missing completion synchronization.

THIS LAB

Lab 5

First the concepts, with code. Then the lab on the server. Process versus thread memory, kernel workers and signals are optional extensions, so they come last.

DISCUSSION

Two workers start, and main returns at once. What happens to them?

Do both finish their work and print? Does main know when they are done?

CONCEPTS FIRST

Processes, threads and joining

Three ideas: what a thread shares, how you wait for it, and what waiting does not do.

A process and a thread

	Process	Thread
What it is	A running program with its own resources	A line of execution inside a process
Address space	Its own: memory is private	Shared with the other threads of the process
Created by	`fork()`	`pthread_create()`
Waited for with	`waitpid()`	`pthread_join()`
Cost to create	Higher	Lower
Risk	Needs a channel to share data	Threads can overwrite each other's data

After fork, parent and child have separate copies of memory. Threads of one process see the same variables.

What threads share, and what they do not

SHARED

Shared by all threads

Code, global variables, the heap, open files. A change made by one thread is visible to the others.

PRIVATE

Private to each thread

Its own stack with local variables, its registers and its place in the code.

TRADE-OFF

Why it matters

Sharing makes communication cheap, and it also lets two threads change one value at the same time.

Create and join

Function	What it does	Error reporting
`pthread_create`	Starts a new thread that runs a function	Returns an error number, 0 on success
`pthread_join`	Waits until one particular thread has finished	Returns an error number, 0 on success
`strerror(rc)`	Turns an error number into text	Use it instead of errno here

These functions return the error number directly. They do not set errno. Check every return value. Compile with -pthread.

Completion is not protection

COMPLETION

Joining answers: is it finished?

pthread_join waits for one worker to finish. After the join, main may read what that worker wrote.

MUTUAL EXCLUSION

A mutex answers: who goes now?

A lock lets only one thread at a time touch a shared value. A join does not do this.

SCOPE

This lab

Each worker writes its own field, so no lock is needed. Joining is the core. Locks come in a later lab.

LAB 5

Workers that finish before main reports

Alex is making a TechCorp worker program concurrent. Each worker must finish before main reports its result.

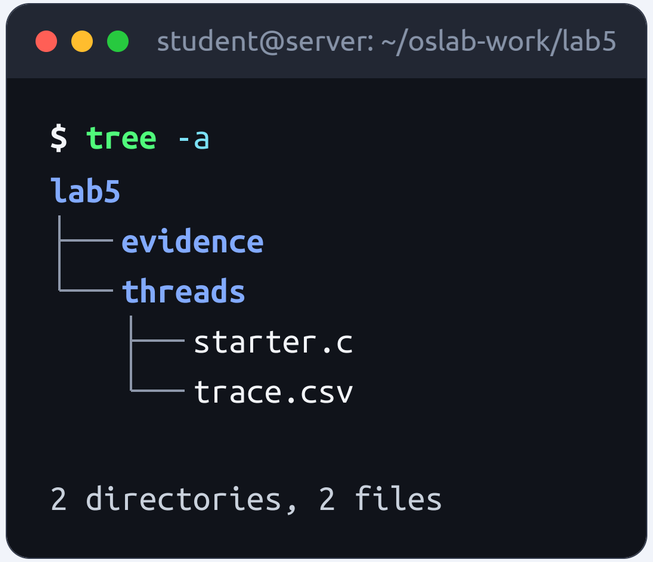
The scenario

SCENARIO

Alex at TechCorp

Make a worker program concurrent. Each worker must finish before main reports its result. Compare thread completion with protecting shared mutable memory.

Every run is bounded by a five-second timeout, and every worker is joined.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Task 1, guided: one thread
25 to 35	Written prediction
35 to 70	Task 2: two workers and completion

Minutes	What happens
70 to 85	Normal and changed runs, feedback
85 to 100	Individual changed-case checkpoint
100 to 110	Correction and explanation
110 to 120	Cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Never signal a kernel worker or another user's process.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

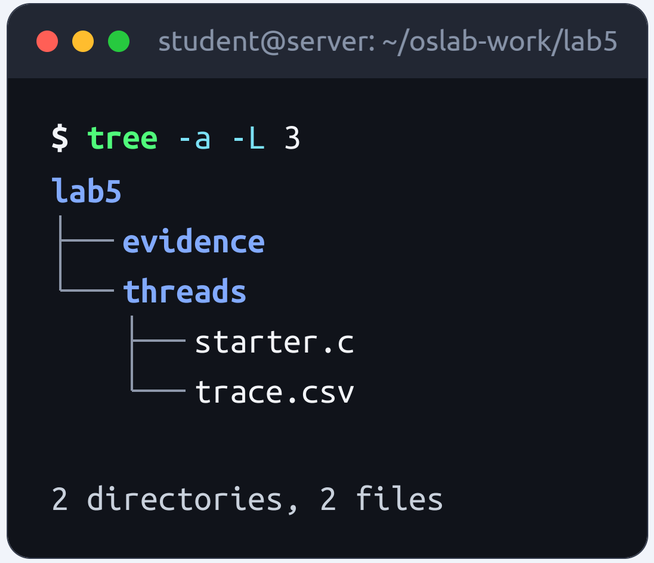
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab5
$ cd "$OSLAB_WORKSPACE/lab5"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab5 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



threads/starter.c is a short two-worker reminder.

threads/trace.csv is an illustration of two workers reading and writing one shared value.

If gcc is missing, do not install anything. Use the source and the printed traces, and say that you did not run the program.

GUIDED

One thread

Task 1, with the instructor: fifteen minutes.

Step 1: check the tools



command -v prints the path of a program, and nothing if it does not exist.

timeout runs a command and stops it after a time limit. Every run in this lab uses it.

CAREFUL

Do not install

Do not install packages on the shared server. Ask the instructor if gcc is missing.

Step 2: the one-worker program

threads/one_worker.c

```c
#include <pthread.h>
#include <stdio.h>
#include <string.h>
static void *worker(void *arg) {
    int *result = arg;
    *result = 6 * 7;
    return NULL;
}
int main(void) {
    int result = 0;
    pthread_t thread;
    int rc = pthread_create(&thread, NULL, worker, &result);
    if (rc) { fprintf(stderr, "create: %s\n", strerror(rc)); return 1; }
    rc = pthread_join(thread, NULL);
    if (rc) { fprintf(stderr, "join: %s\n", strerror(rc)); return 1; }
    printf("result=%d\n", result);
    return 0;
}
```

Save it as threads/one_worker.c. The three marked lines matter most.

1. The worker writes result.
2. Main waits with pthread_join.
3. Main reads result after the join.

The worker gets a pointer to result, so both threads use the same memory.

Step 2: compile and run

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/one_worker.c -o threads/one_worker
$ timeout 5 ./threads/one_worker
result=42
```

-Wall -Wextra turn on warnings. -pthread links the thread library. -o names the program.

pthread_create starts the worker. pthread_join waits for that worker. strerror(rc) gives readable error text.

Step 3: mark three lines

TRY

Mark them on your copy

The line where the worker writes result. The line where main waits. The line where main reads it.

THINK

Observe

Would creating a thread alone tell you when the result is ready? Would a join also stop two workers from racing on one shared counter?

REMEMBER

Compare with fork

After fork, each process would normally have its own private copy of result.

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

If a program starts two workers and main returns without joining them, must both worker-completion messages appear? Explain.

HOW

The rules

Do not run the two-worker program yet. Alone, and without AI. A five-minute comparison with a neighbour is optional, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

TASK 2

Two workers and completion

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1: the starting program, the worker

threads/two_workers.c (1 of 2)

```
struct job { int id; int steps; int result; };
static void *worker(void *arg) {
    struct job *job = arg;
    const struct timespec delay = {0, 20000000};
    for (int i = 0; i < job->steps; ++i) {
        job->result += job->id;
        nanosleep(&delay, NULL); /* teaching delay, not synchronization */
    }
    printf("worker %d finished\n", job->id);
    return NULL;
}
```

Create threads/two_workers.c with the two parts on this slide and the next. The file also starts with the includes and #define _POSIX_C_SOURCE 200809L.

Each worker adds its own id to its own result field, once per step.

The 20 ms delay only makes an early exit by main easier to see. It is not synchronization and it guarantees no order.

Step 1: the starting program, main

threads/two_workers.c (2 of 2)

```
int main(void) {
    struct job a = {2, 3, 0}, b = {3, 4, 0};
    pthread_t ta, tb;
    int rc = pthread_create(&ta, NULL, worker, &a);
    if (rc) { fprintf(stderr, "create A: %s\n", strerror(rc)); return 1; }
    rc = pthread_create(&tb, NULL, worker, &b);
    if (rc) {
        fprintf(stderr, "create B: %s\n", strerror(rc));
        pthread_join(ta, NULL);
        return 1;
    }
    /* TODO: join both workers and check each return code. */
    /* TODO: after both joins, print a.result and b.result. */
    puts("main finishing");
    return 0;
}
```

Main creates both workers. If the second create fails, it joins the first before it leaves.

The two marked TODO lines are your work.

TRY

Run it first

Compile and run it as it is. Does main finishing mean the workers finished?

Step 2 and 3: join both, then print

What to do

1. Join each worker and check each return code, as in the single-worker example.
2. Print a.result and b.result after both joins.
3. Decide which lines belong before main finishing.
4. Compile, read the warnings, and run.

THINK

Why no mutex here?

A should give 6 and B should give 12. Each worker owns a different field, and main reads only after completion.

Step 4: a different design, the trace

The trace shows two workers that each read one shared value, and then write it back.

student@server: ~/oslab-work/lab5

```
$ cat threads/trace.csv
```

THINK

Think

What can go wrong in that design? Would joining the workers after their work prevent the overlapping update?

Complete when, and hints

DONE

Complete when

Both workers complete before main reports results. Creation and join errors are handled. You can tell completion from mutual exclusion.

Hints, one at a time

1. Match one join to each thread that was created.
2. Compare the two thread variables you pass to the joins.
3. Move the result reads after both joins, then ask whether any mutable location is still shared.

Normal run, then a changed run

Fifteen minutes, from minute 70 to 85.

Test 1: a normal run, saved

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/two_workers.c -o threads/two_workers
$ timeout 5 ./threads/two_workers > evidence/normal.txt 2>&1
$ rc=$?; printf 'exit=%s\n' "$rc" >> evidence/normal.txt
$ cat evidence/normal.txt
```

State the expected results, and explain why the order of the two finished messages may vary.

$? is the exit status of timeout. 124 means the time limit ran out, not success.

Test 2: change a worker, predict again

student@server: ~/oslab-work/lab5

```
$ timeout 5 ./threads/two_workers > evidence/changed.txt 2>&1
$ rc=$?; printf 'exit=%s\n' "$rc" >> evidence/changed.txt
$ cat evidence/changed.txt
```

Change B to two steps. Predict its new result: 6. Recompile with the same gcc line, then run.

Also try zero steps for one worker. It must still be joined, even if it finishes at once.

What each run shows

Run	Expected	What it tests
Normal	A is 6, B is 12, exit=0	Creation, joins, results read after completion
B with two steps	B is 6	That the result follows the steps, not a timing accident
A with zero steps	A is 0, and still joined	That a fast worker needs a join as well
Exit status 124	Not a success	That the time limit expired

Repeated successful runs do not prove there is no race. Without -pthread the build can fail. If gcc is missing, report that the behaviour was not tested.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

Main joins worker A but not worker B, then reads B's result. Explain what remains unestablished and name the smallest synchronization change. Would that change alone protect a counter that both workers update at the same time?

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) What makes the result read safe after joining? (b) Why is the delay not a lock? (c) What would change if both workers updated one field?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab5/evidence"
$ cd "$OSLAB_WORKSPACE/lab5"
$ cp -- threads/two_workers.c "$SUBMISSION_REPO/lab5/"
$ cp -- evidence/normal.txt evidence/changed.txt "$SUBMISSION_REPO/lab5/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the source, not the compiled program, with the two evidence files. Make sure every program has ended.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab5
├── evidence
│   ├── changed.txt
│   └── normal.txt
├── README.md
└── two_workers.c

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Correct worker creation, joins and result use	3
Normal and changed-step or zero-step evidence, with the build diagnosed	2
Thread memory, completion and mutual exclusion explained; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
No -pthread in the gcc line	A build or link error	Add -pthread
Reading a result before the join	An old value, sometimes right	Move the reads after both joins
Joining only one worker	A message that sometimes goes missing	Join every thread that was created
Trusting the delay as a lock	It works until the timing changes	Join, and use a mutex for shared writes

BEYOND THE CORE

Fork, kernel workers and signals

Optional extensions. Each uses a bounded run and your own processes.

Fork copies memory, threads share it

a small fork() program

```
$ ./fork_demo
child: value=9 pid=4242
parent: value=1 pid=4241
```

Set an integer to 1. Change it to 9 only in the child. The parent waits with waitpid, then prints.

The parent still sees 1: after fork the two processes have separate memory. Threads would see 9.

Check fork and waitpid for errors, flush output before fork, and run under timeout 5.

Looking at threads from outside

student@server: ~/oslab-work/lab5

```
$ ./threads/two_workers & pid=$!
$ ps -L -p "$pid" -o pid,lwp,nlwp,comm
$ wait "$pid"
$ ps -eo pid,ppid,comm | head -n 20
```

ps -L lists each thread: LWP is the thread's ID, and NLWP counts the threads of the process.

Raise the worker delay to about two seconds per step, so you have time to look.

Kernel workers may be hidden inside a container or WSL. Not seeing them does not prove they are absent. Observe them only; never signal them.

A signal handler that does almost nothing

signal_demo.c

```
static volatile sig_atomic_t stop;
static void on_term(int sig) {
    (void)sig;
    stop = 1;  /* only set a flag */
}
/* in main: register it, then loop */
struct sigaction sa = {0};
sa.sa_handler = on_term;
sigaction(SIGTERM, &sa, NULL);
while (!stop) { /* work */ }
/* print and clean up here, after the loop */
```

The handler only sets a flag. Printing and cleanup happen in the normal loop, not inside the handler.

Keep a ten-second limit. Start it, note its own PID, and send kill -TERM to that PID only.

A join waits for completion. A mutex protects shared data. A signal flag is neither.

Key takeaways

01

Thread or process

Threads share memory in one process. Forked processes get separate copies.

02

Create, then join

Check every return code, and join each thread you created.

03

Completion is not protection

A join says it finished. A mutex says who may write now.

04

Bound every run

timeout 5 keeps a mistake from running forever. Exit 124 means it ran out of time.

Next session

COMING UP

Lab 6: users, groups and file permissions

Who may read, write or run a file, and how Linux decides.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 4, Threads and Concurrency.